Midnight Grader - CS193 Homework 9 Design Document

Johaan Joseph Mannanal | November 1, 2026

Status: My plan and system diagram are ready. I haven't started the Java code yet.

1.1 Problem statement

I want to help students check their practice quizzes without having to compare every answer manually. It takes time, and it's easy to miss a wrong answer.

1.2 Idea / solution

I'm going to build a Java program that students can run in the terminal to check a multiple-choice practice quiz. It will read their answers and an answer key from CSV files, show their score and missed questions, and save the results.

2.1 Eisenhower Matrix and MVP

	Urgent	**Not urgent**
Important	**Do — MVP** ● Load an answer key and student answers. ● Compare answers and calculate a score. ● Display and export the results.	**Schedule — Later** ● Grade multiple quizzes in one run. ● Support questions worth different points. ● Show results from previous attempts.
Not important	**Delegate — Use existing libraries** ● File reading and writing through Java’s standard library. ● Console input through Scanner. ● Percentage formatting through Java’s formatting tools.	**Delete — Outside this project** ● Handwriting recognition. ● AI grading of essays. ● Accounts and cloud syncing.

Basic grading rules

- Each question is worth one point.
- A blank or missing answer counts as incorrect and is reported as unanswered.
- Match answers by question number, not by row order.
- Report missing files or malformed input clearly.

2.2 Tech stack

I'll write it in Java using the same JDK I use for CS180.

The program will run in the terminal.

I'll use Java's standard library to read and write files, take input with Scanner, and format scores. I won't need any extra libraries.

I'll keep the answer key, student answers, and results in local CSV files.

I'll use sample quiz files for testing. The program won't need the internet or an API.

I'll use IntelliJ to write and run the code, and Git and GitHub to track my progress.

Input format

Both files will start with question,answer. Each row will have a question number and an answer. A student can leave an answer blank. I've added sample files in examples/.

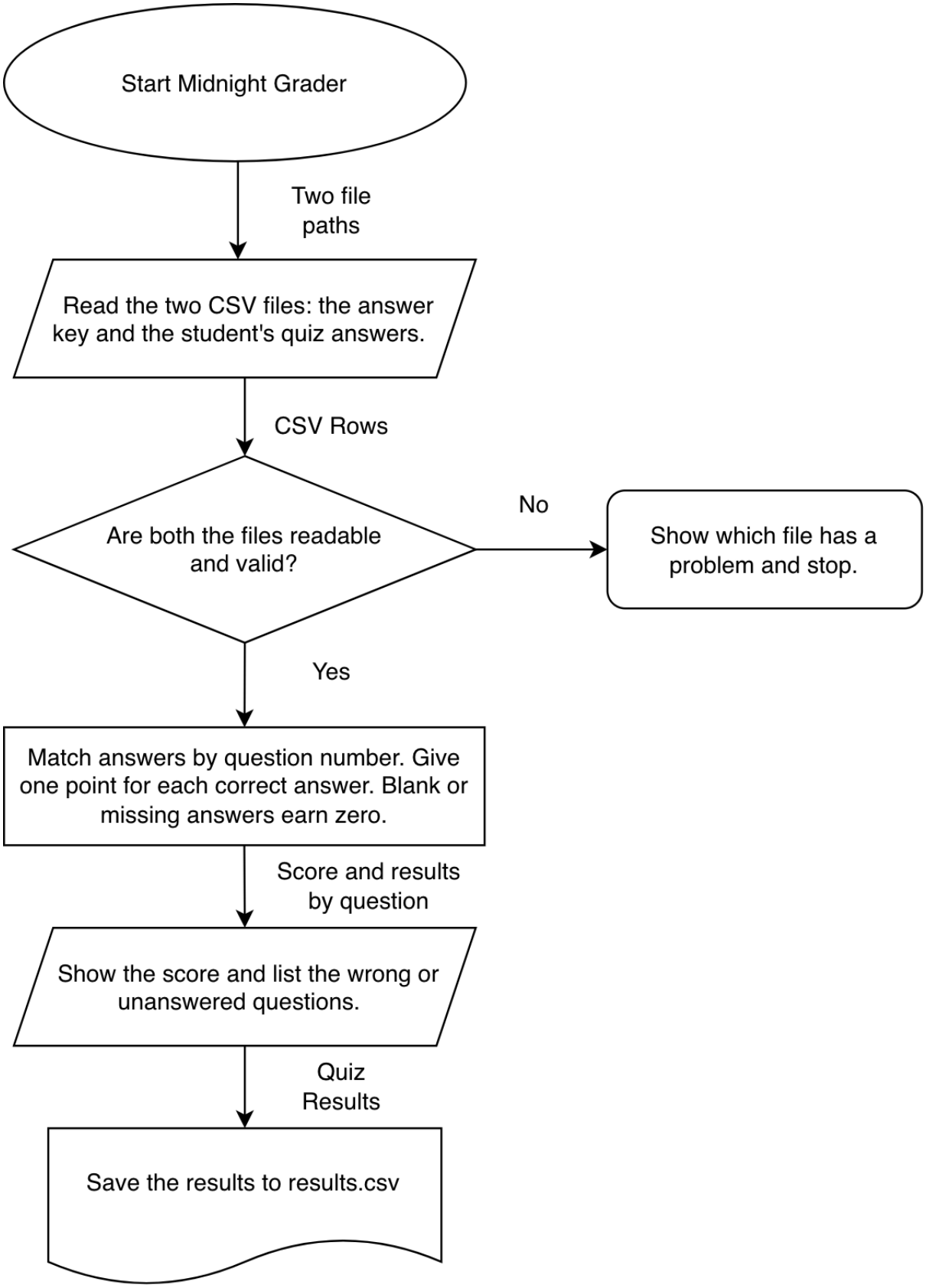
2.3 Chunked build plan

1. Set up my GitHub repo, project folders, README, and a few sample CSV files.
2. Get the program to read an answer key and report problems with the file.
3. Add support for reading a student's answers, including blank answers.
4. Compare the answers by question number and work out the score.
5. Show the score and which questions were wrong or left blank.
6. Save those results to a CSV file.
7. Try missing files, empty files, and bad input, then finish the README instructions.
8. Add batch grading on a separate branch so I can check several quizzes in one run. Test it and merge it once it works.

I'll test each step before moving on and make a commit that explains what changed. I'll build batch grading on a separate branch and check that both versions work before merging it.

2.4 System diagram

The grader checks both CSV files before scoring the quiz.



3.1 GitHub repository and README

Public repository: https://github.com/Johaan-Mannanal/midnight-grader-java

Homework Google Doc: https://docs.google.com/document/d/1RyccEZwJ1W4BKwszNWhDETVaH8-iuPGLJmULnyaFADA/edit

Folder structure: `src/` for future Java code, `tests/` for future tests, `docs/` for planning and the student-drawn diagram, `examples/` for synthetic input CSV files, and `output/` for generated reports.

The README explains what I'm building, what goes into the first version, and what I want to add later. It also lists the tools and file format. I'll add instructions for compiling and running the program when the Java code is ready.